# COMP6210/3210
# Big Data

## Week 10: Workshop

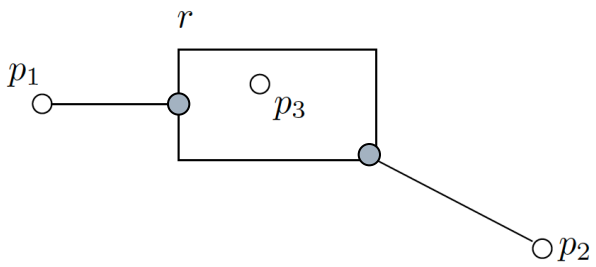
School of Computing

Macquarie University

# NN Search

## Mindist

Given a point $q$ and an axis-parallel rectangle $r$, the *mindist* of $q$ and $r$, denoted as $mindist(q, r)$, equals $\min_{p \in r} \|q, p\|$.




In the above example, with respect to $r$, the mindists of $p_1$ and $p_2$ are equal to the lengths of the two segments shown, while that of $p_3$ is 0.

Think: how to compute $mindist(q, r)$ in $O(d)$ time?

# Multiple Choice Questions

1. Which of the following will be the Euclidean Distance between the two data points A(1,3) and B(2,3)?

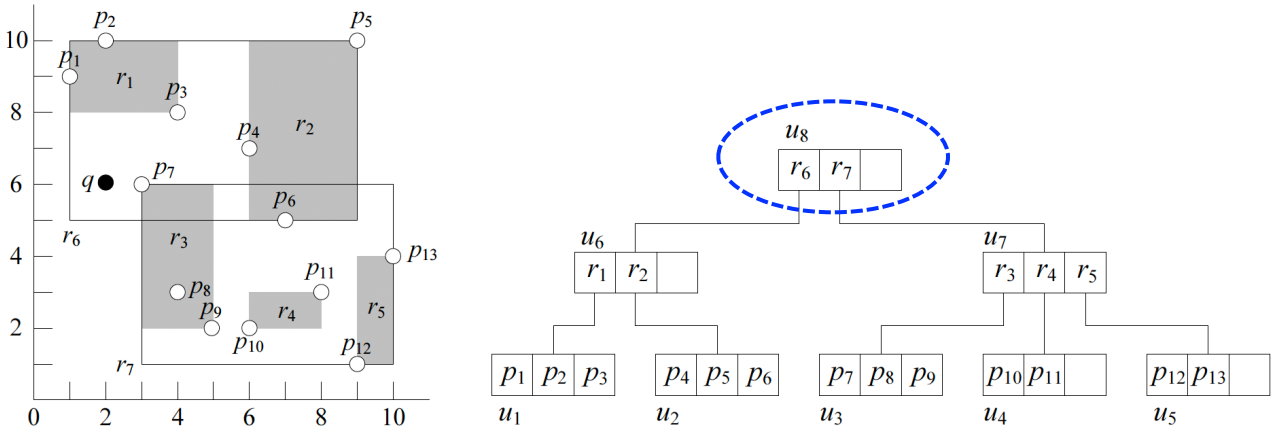
- 1
- 2
- 3
- 4

# Branch and Bound (BaB) Algorithm

Algorithm 1: Branch-and-bound (BaB)

BaB performs a depth-first traversal of the R-tree but uses mindists to (i) prioritize the nodes for accessing, and (ii) prune the nodes that cannot contain the final answer.

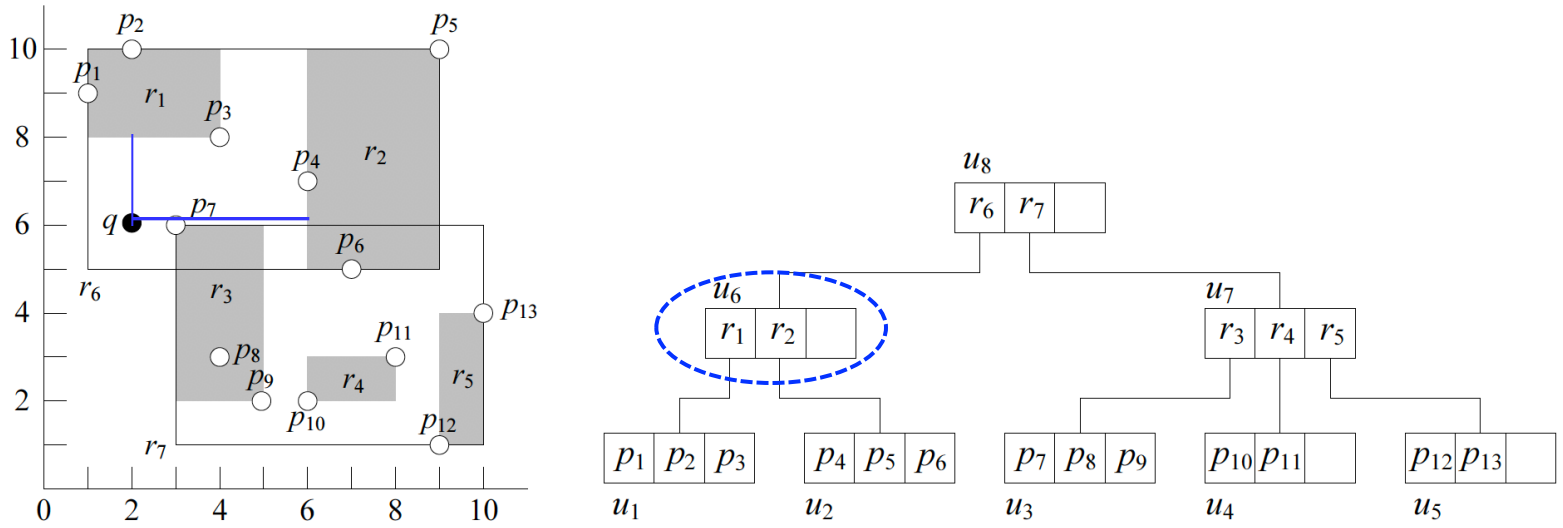
Let us illustrate the algorithm from an example. To find the NN of $q$ (as shown in the figure), BaB starts from the root of the R-tree, where it sees two MBRs $r_6$ and $r_7$. The mindists from $q$ to $r_6$ and $r_7$ are 0 and 1, respectively. Since $mindist(q, r_6)$ is smaller, algorithm visits $u_6$ next.



# Branch and Bound (BaB) Algorithm

## Branch-and-bound (BaB)

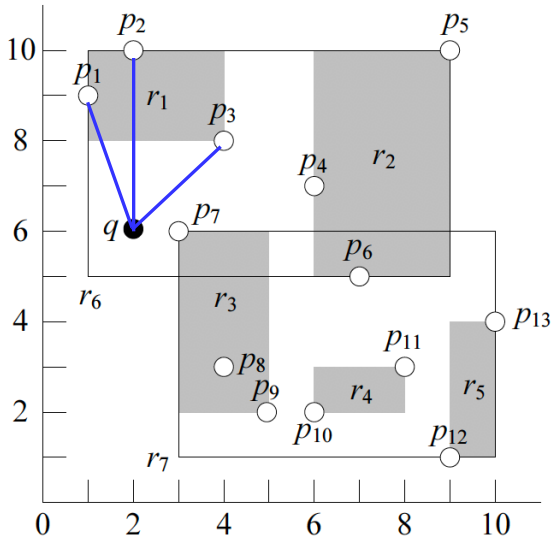
At node $u_6$, BaB chooses to descend into MBR $r_1$, because its mindist from $q$ is smaller than that of $r_2$.



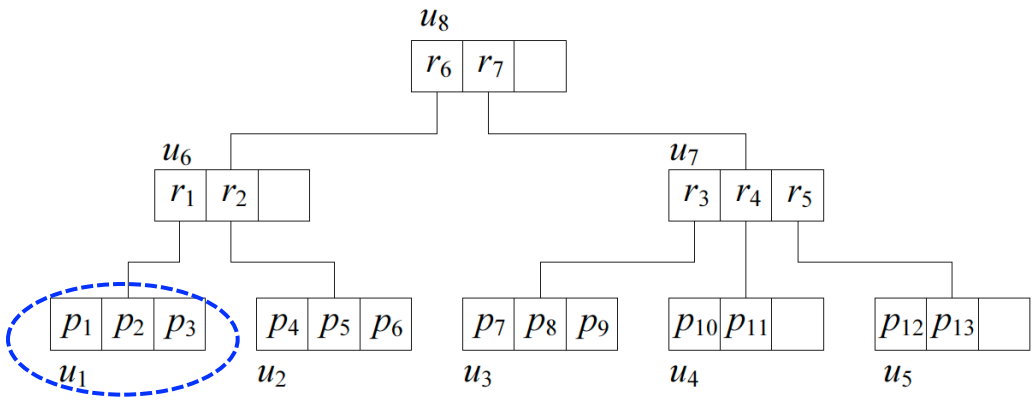
# Branch and Bound (BaB) Algorithm

## Branch-and-bound (BaB)

Now the algorithm is at the leaf node $u_1$. It simply computes the distance from $q$ to each data point in $u_1$, and remembers the nearest one, i.e., $p_3$. This is the current NN of $q$ found so far.



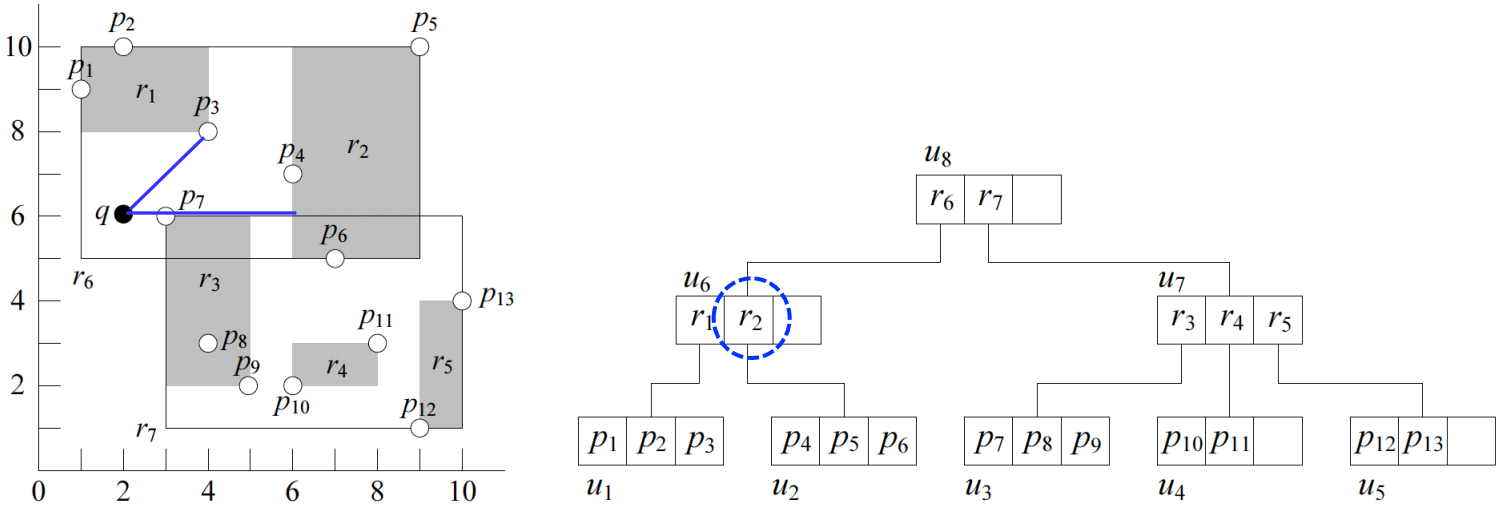

*$p_3$ is the result or not?*



# Branch-and Bound (BaB) Algorithm

## Branch-and-bound (BaB)

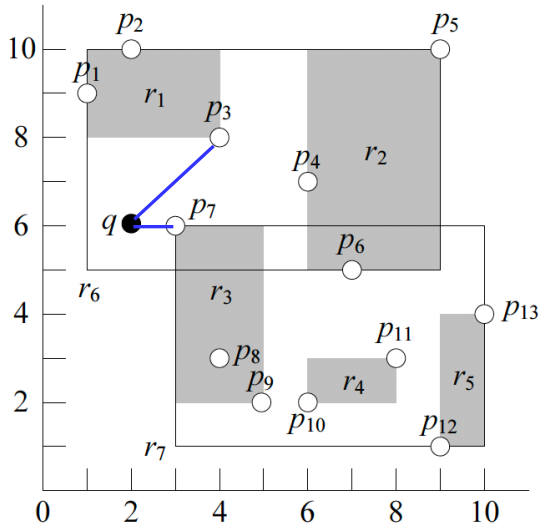
Now the algorithm backtracks to node $u_6$, where the subtree of MBR $r_2$ has not been explored yet. However, the fact that the $mindist(q, r_2) = 4$ is greater than the distance $2\sqrt{2}$ from $q$ to the current NN $p_3$ rules out the possibility that the NN of $q$ can be inside $r_2$. Therefore, the subtree of $r_2$ can be pruned.

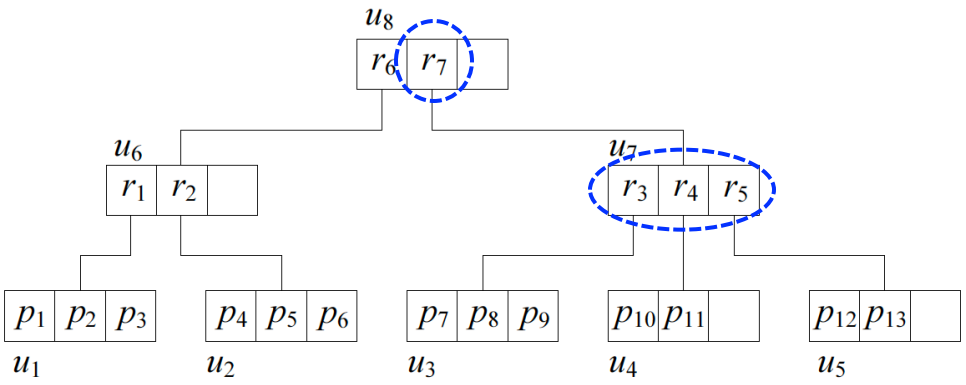


# Branch-and Bound (BaB) Algorithm

## Branch-and-bound (BaB)

Now we backtrack to the root, where MBR $r_7$ has not been processed yet. The mindist 1 between $q$ and $r_7$ is smaller than $\|q, p_3\| = 2\sqrt{2}$. Therefore, the child $u_7$ of $r_7$ must be visited.



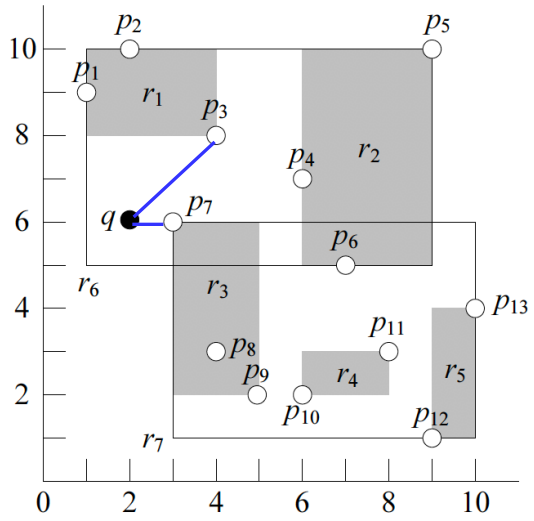



# Branch-and Bound (BaB) Algorithm

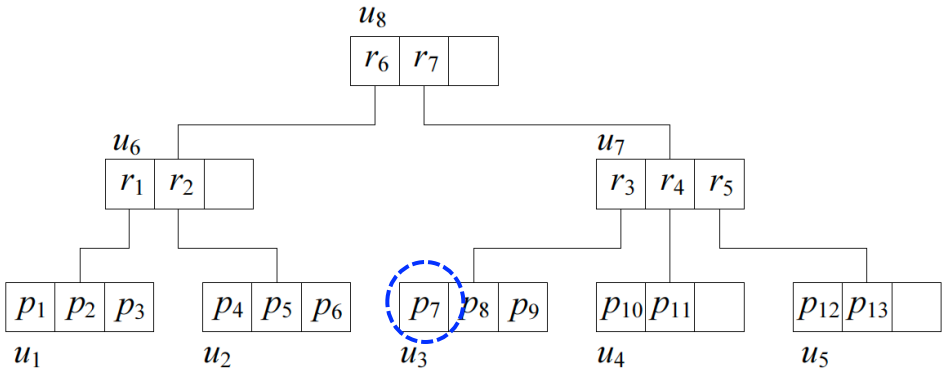
## Branch-and-bound (BaB)

At node $u_3$, BaB finds $p_7$ which replaces $p_3$ as its current NN.

Then, it backtracks to node $u_7$ and prunes $r_4$ and $r_5$. After that, the algorithm backtracks one more level to the root. As all the MBRs of the root have been processed, it terminates with $p_7$ as the final result.




*We prune $r_2$, $r_4$, and $r_5$. Namely, we do not access the data points included in the three MBRs.*



# Multiple Choice Questions

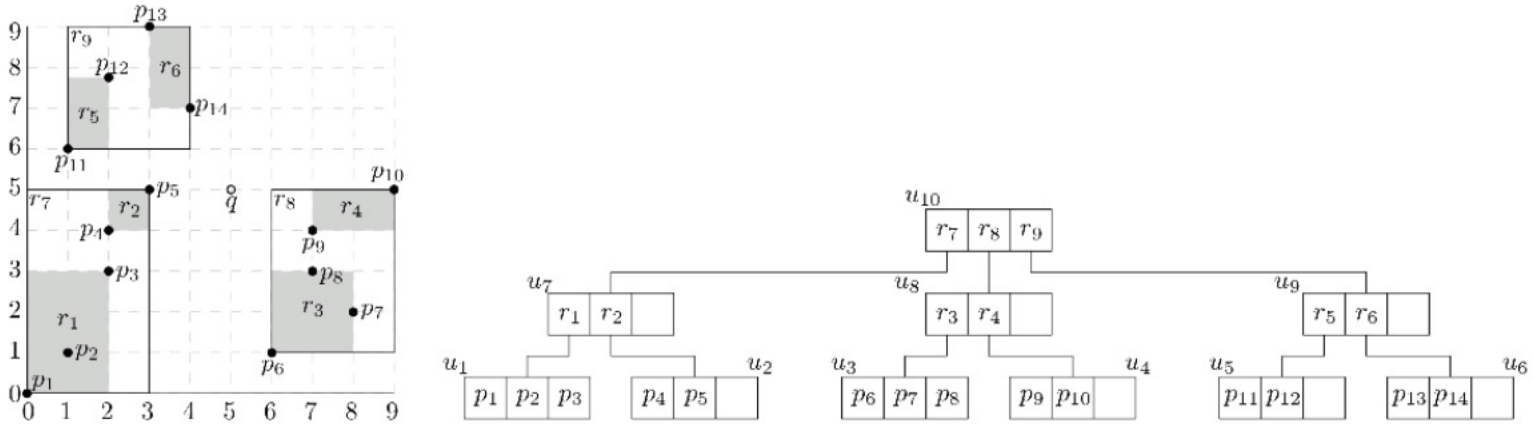
2. Which of the following is NOT true for BaB Nearest Neighbour Search Algorithm?

- It starts from the root.
- It does not have to access all the data points.
- It keeps a sorted list for the whole tree.
- It needs to backtracks the nodes that are not visited yet.

# Tutorial Questions

## Part 1: Nearest Neighbor Search

**Problem 1.** Consider the dataset $P = \{p_1, \ldots, p_{14}\}$, an R-tree on $P$, and a nearest neighbor query point $q$ as shown below. Indicate the nodes accessed by the BaB algorithm.

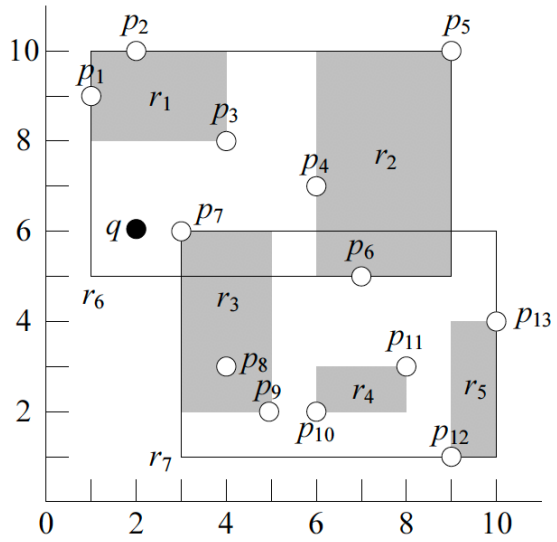


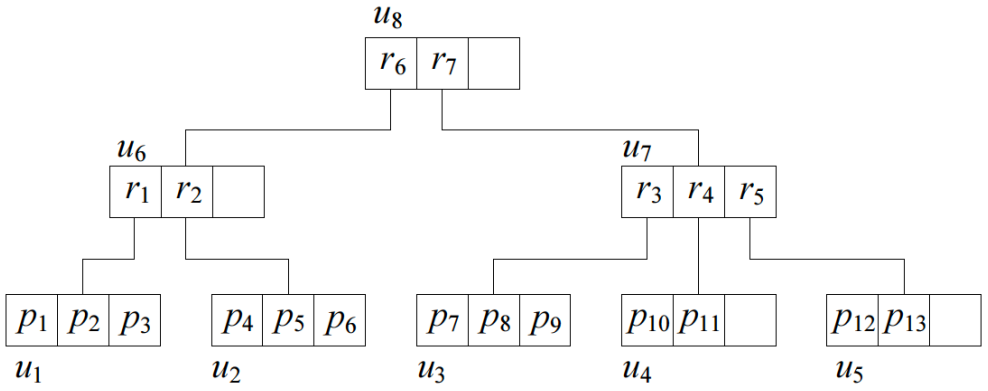

**Problem 2.** Indicate the nodes accessed by the BF algorithm in the example of the previous problem.

# Best First (BF) Algorithm

## Algorithm 2: Best First (BF)

We have seen that BaB accessed $u_8, u_6, u_1, u_7, u_3$. Next, we will learn a better algorithm called best first (BF) that can avoid accessing $u_1$.

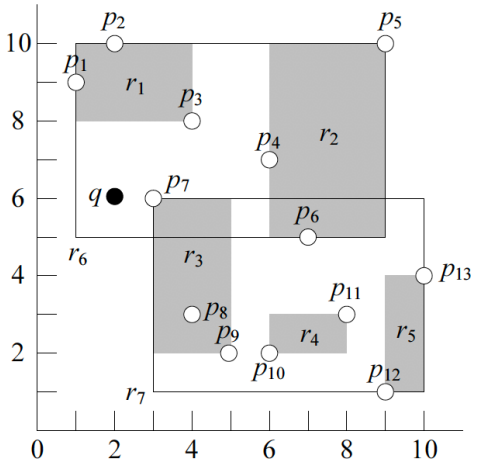


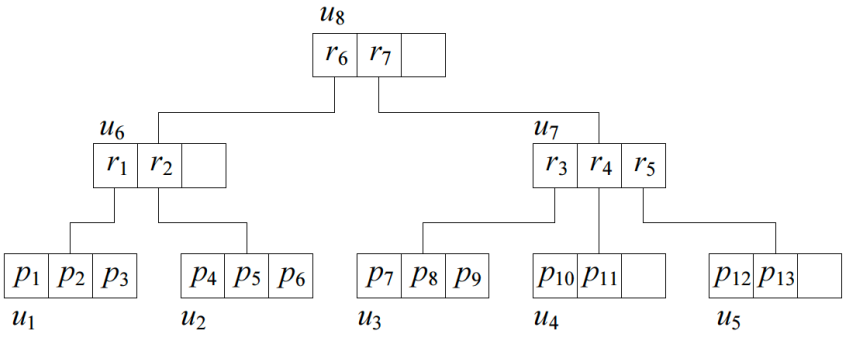



# Best First (BF) Algorithm

**Algorithm 2: Best First (BF)**

Again, we illustrate the BF algorithm with an example. As with BaB, BF also starts from the root. At any moment, the algorithm keeps in memory all the intermediate MBRs that have been seen but not yet accessed in a sorted list $H$, using their mindists to $q$ as the sorting keys. In our example, so far we have seen only two MBRs $r_6, r_7$, so $H$ has two entries $\{(r_6, 0), (r_7, 1)\}$.



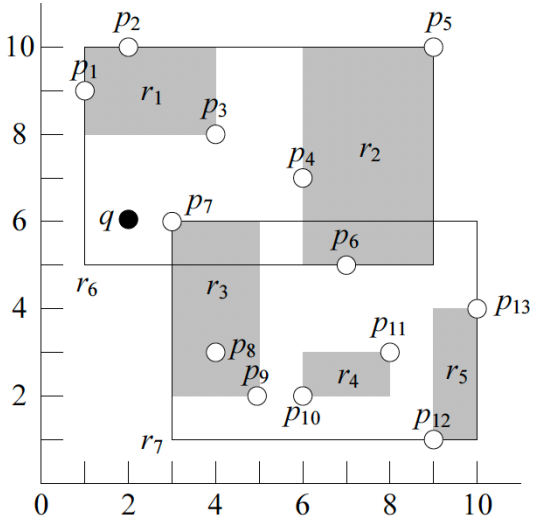



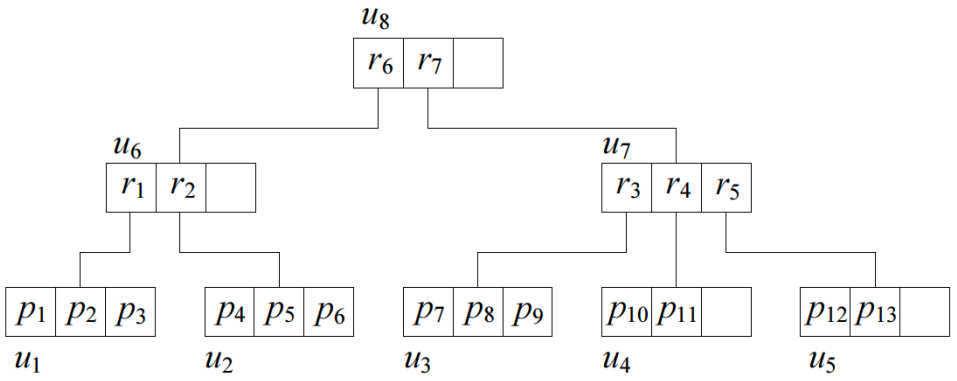
# Best First (BF) Algorithm

## Best First (BF)

Each iteration of BF removes from $H$ the MBR with the smallest mindist, and accesses its child node. Continuing the example, BF removes $r_6$ from $H$, visits its child node $u_6$, and adds to $H$ the MBRs $r_1, r_2$ there. At this time, $H = \{(r_7, 1), (r_1, 2), (r_2, 4)\}$.

$\{(r_6, 0), (r_7, 1)\}$

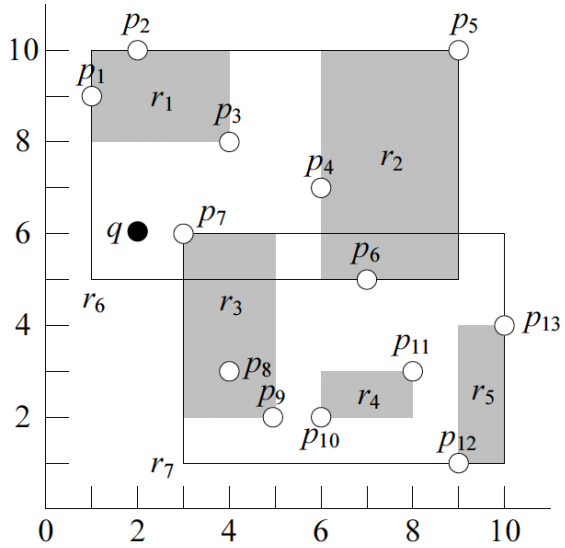


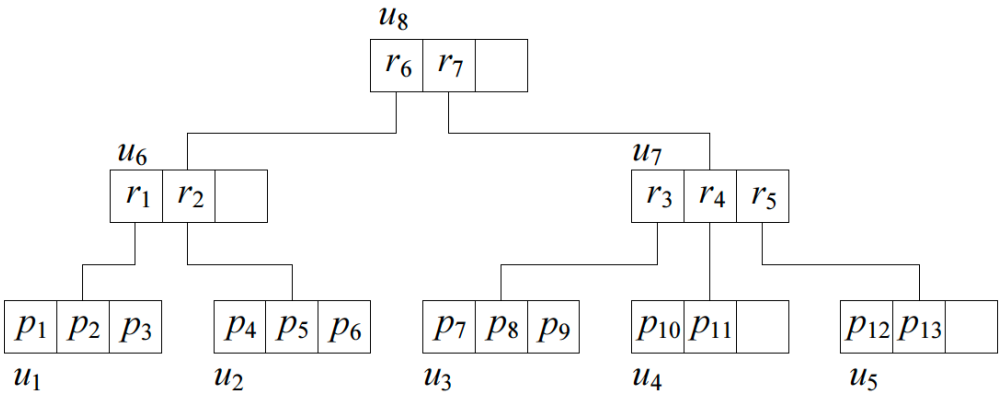



# Best First (BF) Algorithm

## Best First (BF)

Similarly, as $r_7$ has the smallest key in $H$, BF accesses its child node $u_7$, after which $H = \{(r_3, 1), (r_1, 2), (r_2, 4), (r_4, 5), (r_5, \sqrt{53})\}$.



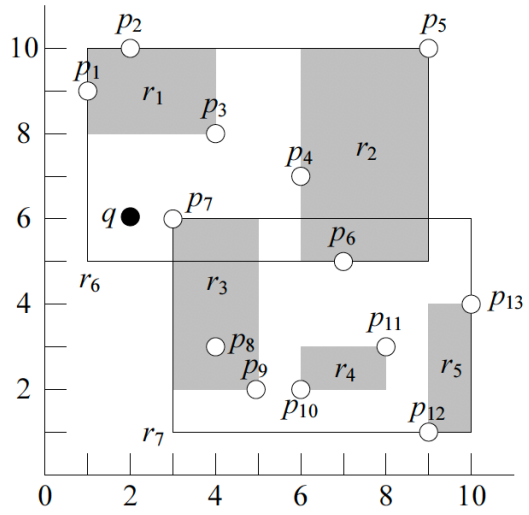



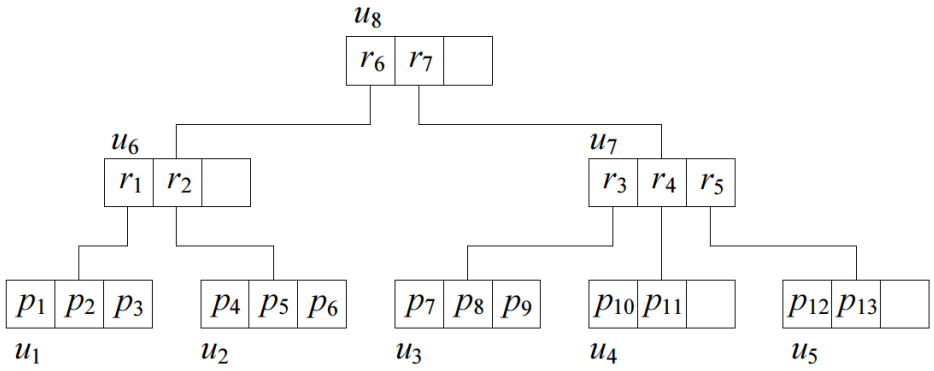
# Best First (BF) Algorithm

## Best First (BF)

Next, the algorithm visits leaf node $u_3$, where $p_7$ is taken as the current NN. Then, BF terminates because $\|q, p_7\| = 1$ is smaller than the lowest mindist of the MBRs in $H = \{(r_1, 2), (r_2, 4), (r_4, 5), (r_5, \sqrt{53})\}$, implying that $p_7$ must be the final NN.

*avoid accessing $p_1$, $p_2$, and $p_3$*



# Multiple Choice Questions

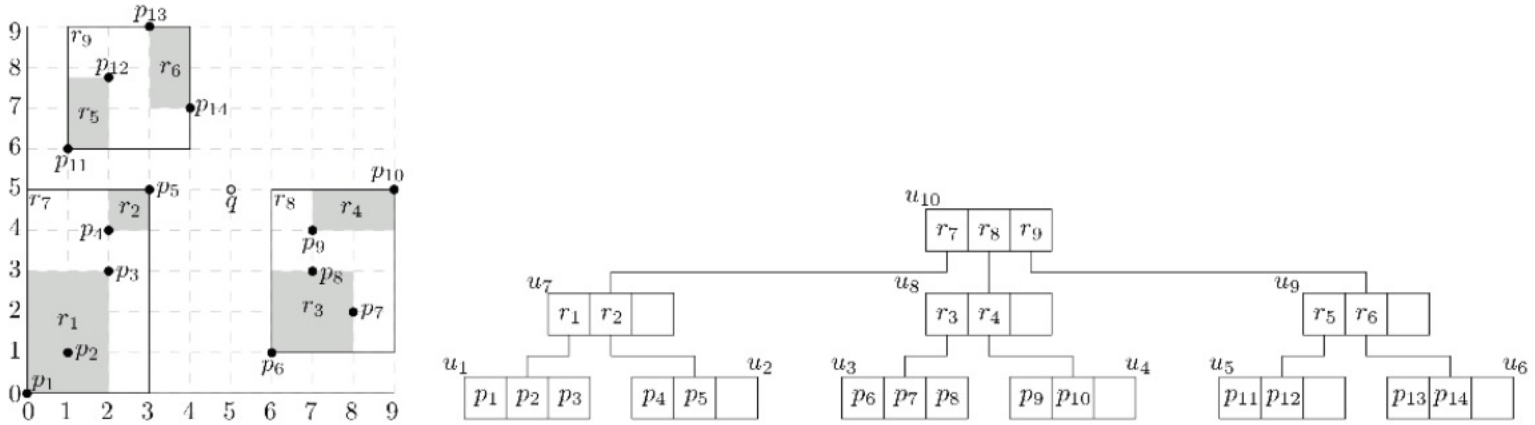
3. Which of the following is NOT true for BF Nearest Neighbour Search Algorithm?

- It starts from the root.
- It can prune more MBRs than BaB Algorithm.
- It keeps a global sorted list.
- It does not perform backtrack procedure.

# Tutorial Questions

## Part 1: Nearest Neighbor Search

**Problem 1.** Consider the dataset $P = \{p_1, \ldots, p_{14}\}$, an R-tree on $P$, and a nearest neighbor query point $q$ as shown below. Indicate the nodes accessed by the BaB algorithm.

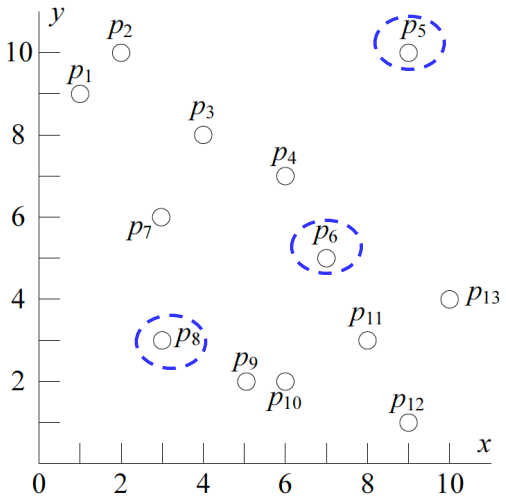



**Problem 2.** Indicate the nodes accessed by the BF algorithm in the example of the previous problem.

# Skyline

**Dominance**

A point $p_1$ dominates $p_2$ if the coordinate of $p_1$ is smaller than or equal to $p_2$ in all dimensions, and strictly smaller in one dimension.

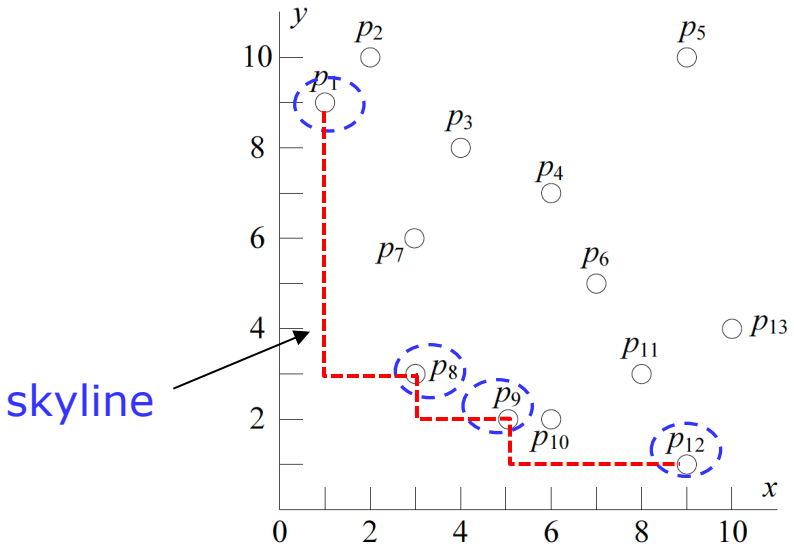



For example, $p_8$ dominates $p_6$, which dominates $p_5$. It is clear that dominance is transitive.

# Skyline

**Skyline Search**

Let $P$ be a set of $d$-dimensional points in $\mathbb{R}^d$. The skyline of $P$ is the set of points in $P$ that are not dominated by others.




$p_7$ is dominated by $p_8$
$p_{10}$ is dominated by $p_9$

The skyline is $\{p_1, p_8, p_9, p_{12}\}$.

# NN Based Skyline Search

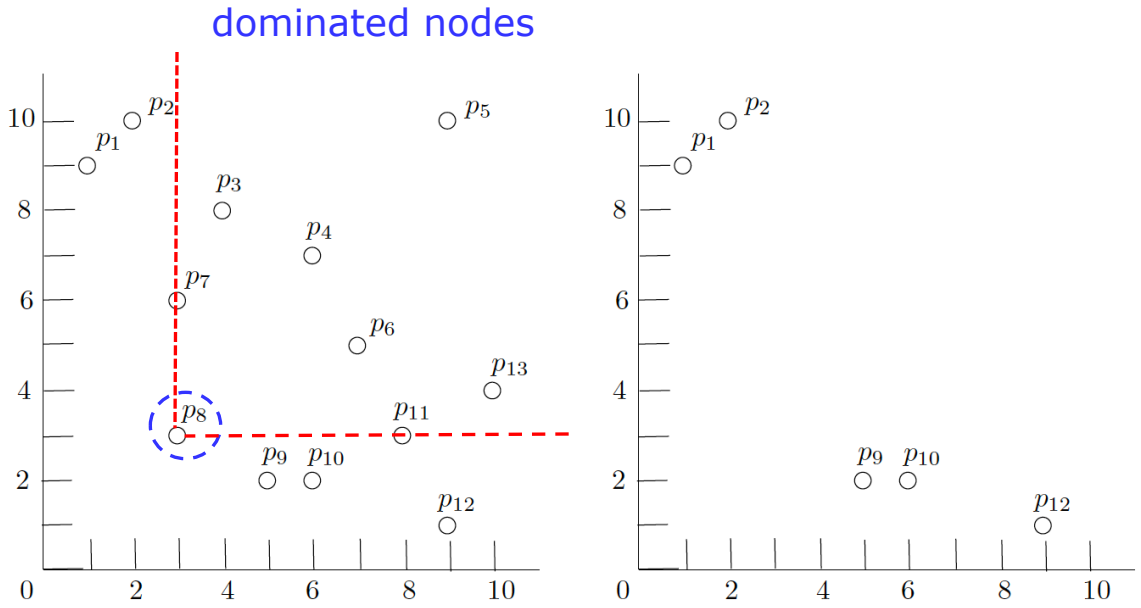
## NN Search as a Black Box

Motivated by the previous observation, we can find the skyline by repeating the next two operations until the R-tree is empty:

- Find the NN $p$ of the origin of the data space. Include $p$ into the skyline.
- Delete $p$ from the tree, as well as all the points that are dominated by $p$.

# NN Based Skyline Search

Example

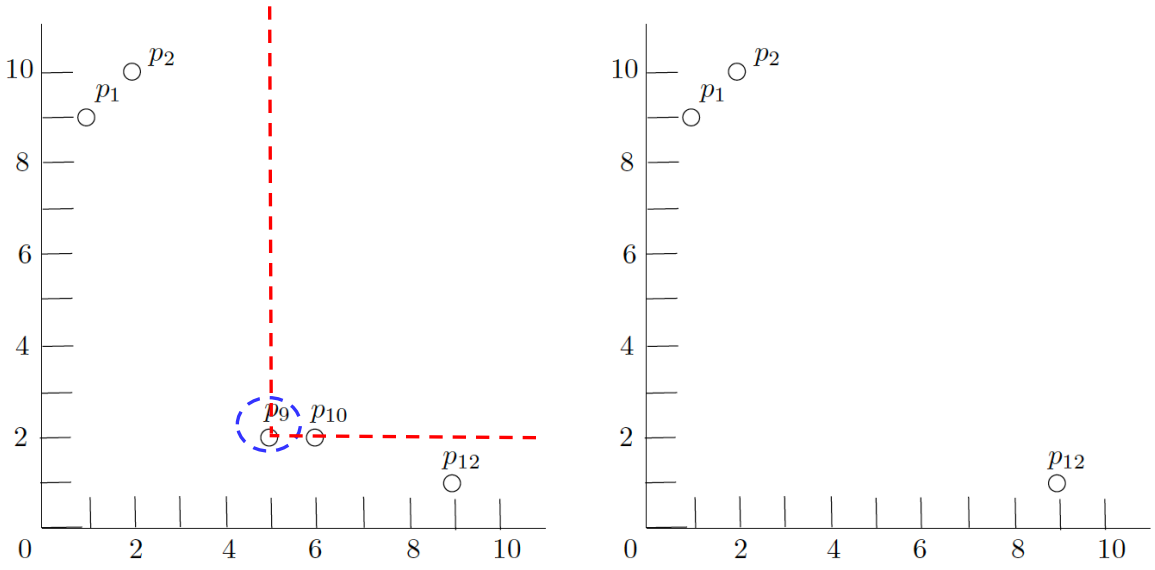



The first NN query finds $p_8$. The figure on the right shows the dataset after the corresponding deletions.

Skyline points={$p_8$}

# NN Based Skyline Search

Example

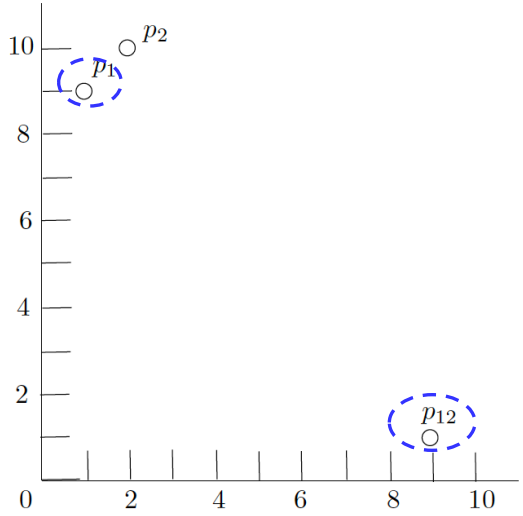



The second NN query finds $p_9$. The figure on the right shows the dataset after the corresponding deletions.

Skyline points={$p_8$, $p_9$}

# NN Based Skyline Search

Example

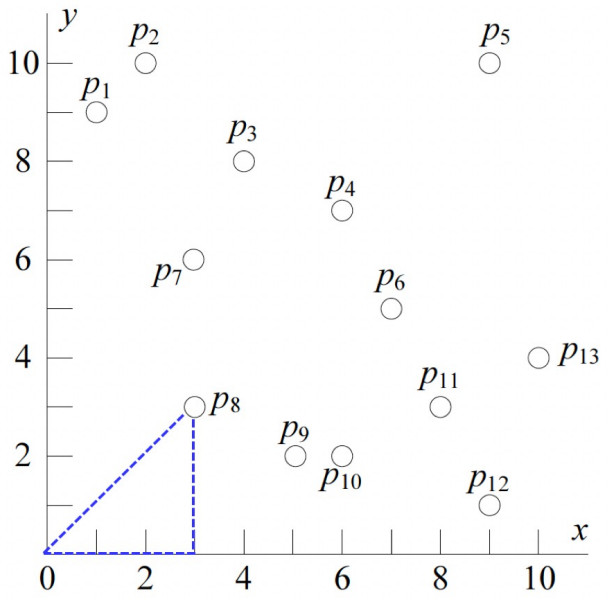



The third NN query finds $p_1$ and $p_{12}$. The dataset becomes empty after corresponding deletions.

Final skyline point $= \{p_8, p_9, p_1, p_{12}\}$.

# NN Based Skyline Search

This previous algorithm must access all the points in the dataset (why?). The next algorithm remedies the defect by accessing (typically) only a part of the dataset.




E.g., If $p_{12}$ is included in another MBR, we cannot prune that MBR by using Branch and Bound NN algorithm.

We have to access all the points to determine the domination relations and remove the corresponding points that are dominated.

# Multiple Choice Questions

4. Which of the following is NOT true for Skyline search?

- (2, 3) dominates (4, 3)
- NN-based Skyline algorithm needs to access all the data points
- BBS Skyline algorithm needs to backtracks the nodes that are not visited.
- BBS Skyline algorithm does not need to access all the data points

# Branch and Bound Skyline

Second Algorithm: Branch and Bound Skyline (BBS)

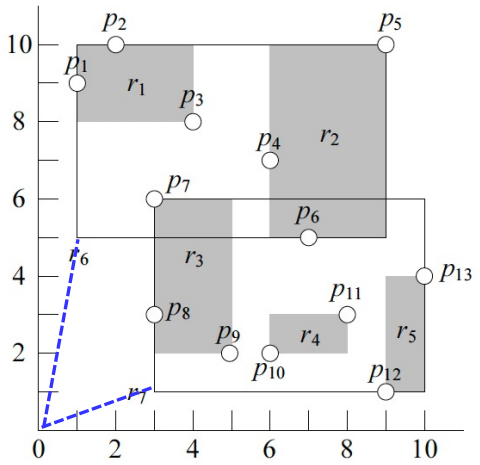
The BBS algorithm can be thought of a variation of the BF algorithm for NN search. It accesses the nodes of the R-tree in ascending order of their mindists from the origin. Different from NN search, however, once an MBR in its entirety is dominated by a skyline point already found, it can be pruned.

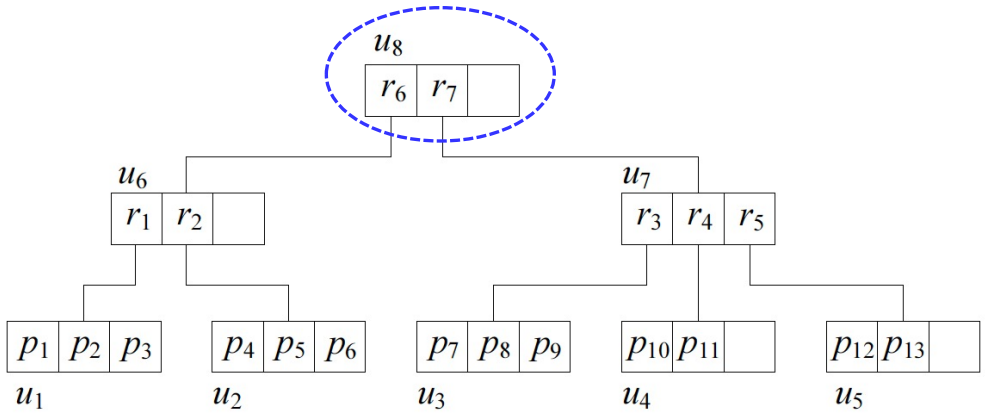
Next we will illustrate the algorithm through an example.

# Branch and Bound Skyline

BBS

First, we access the root, and put the MBRs there in a sorted list $H$, namely, $H = \{(r_7, \sqrt{10}), (r_6, \sqrt{26})\}$. Note that the sorting key of its MBR is its mindist to the origin.

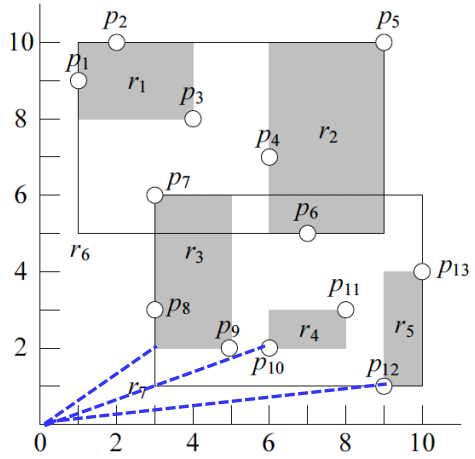


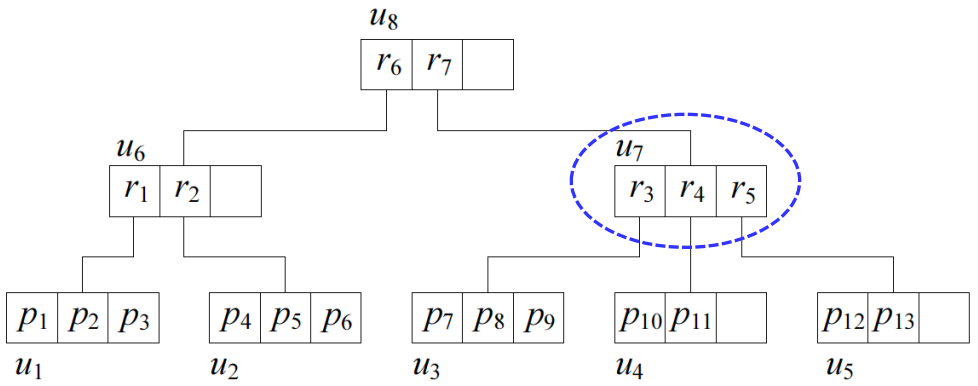



# Branch and Bound Skyline

BBS

The algorithm then visits node $u_7$, after which $H = \{(r_3, \sqrt{13}), (r_6, \sqrt{26}), (r_4, \sqrt{40}), (r_5, \sqrt{82})\}$.

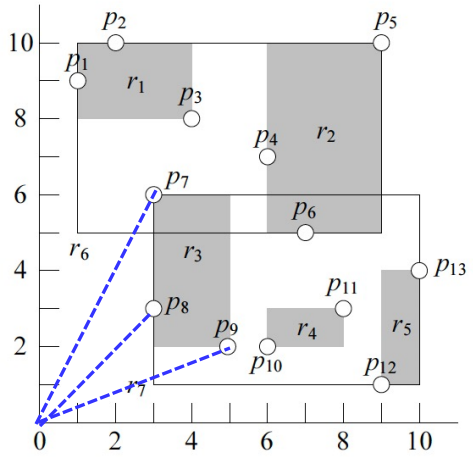


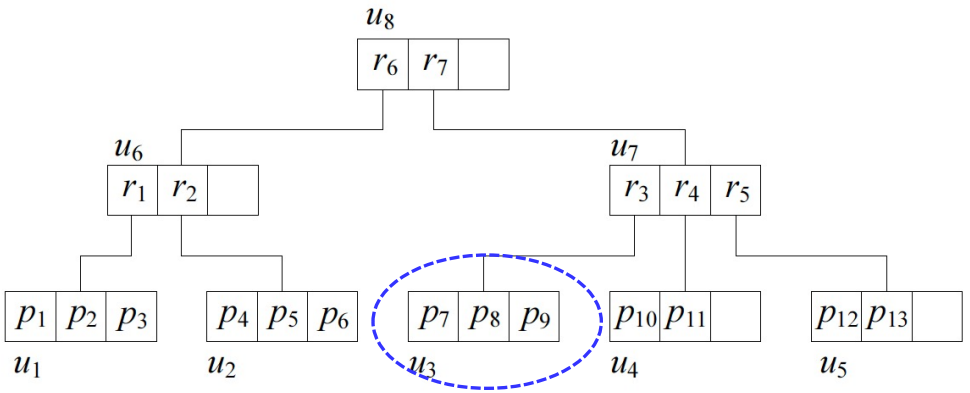



# Branch and Bound Skyline

BBS

We now visit $u_3$ which is a leaf node. Insert all the points therein into $H$, which becomes:
$H = \{(p_8, \sqrt{18}), (r_6, \sqrt{26}), (p_9, \sqrt{29}), (r_4, \sqrt{40}), (p_7, \sqrt{45}), (r_5, \sqrt{82})\}$.



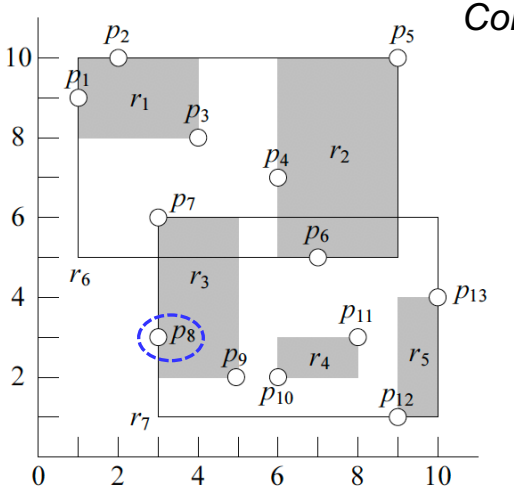



# Branch and Bound Skyline

BBS

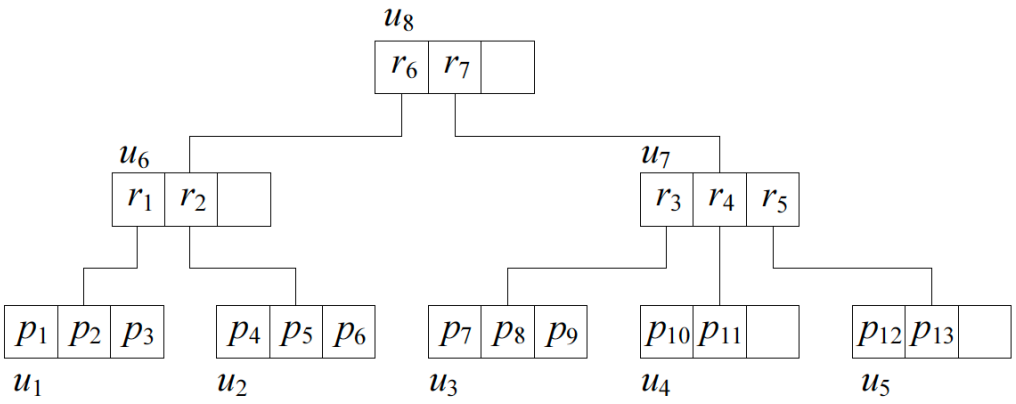
Now we remove $p_8$ from $H$. This is the first skyline point found.

$H$ is now $\{(r_6, \sqrt{26}), (p_9, \sqrt{29}), (r_4, \sqrt{40}), (p_7, \sqrt{45}), (r_5, \sqrt{82})\}$.




*Compare p8 with the lower-left corner of $r_6$*

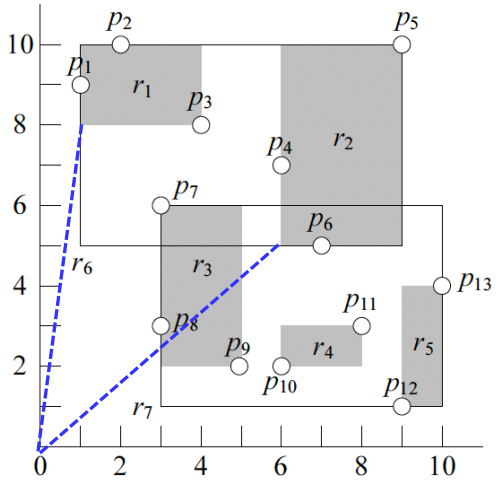
$(p_8, \sqrt{18})$

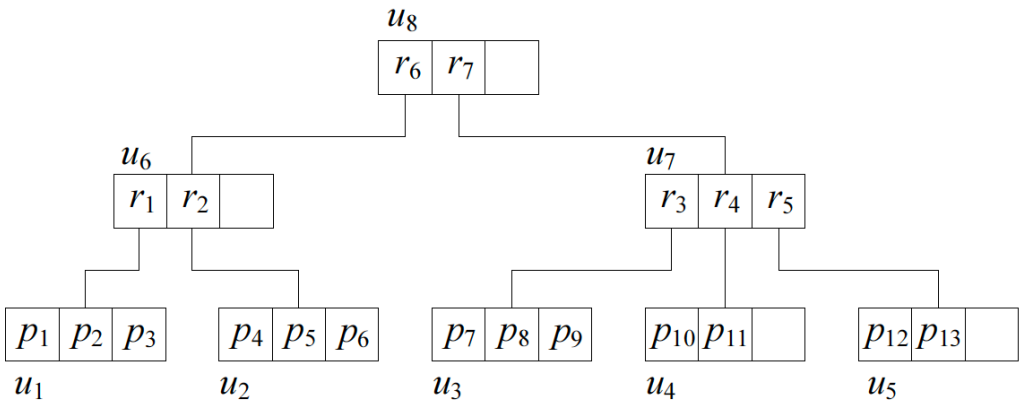


# Branch and Bound Skyline

BBS

Next, access $u_6$, and update $H$ to
$\{(p_9, \sqrt{29}), (r_4, \sqrt{40}), (p_7, \sqrt{45}), (r_2, \sqrt{61}), (r_1, \sqrt{65}), (r_5, \sqrt{82})\}$.



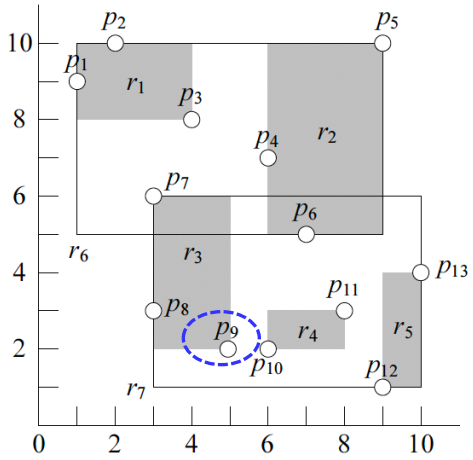
# Branch and Bound Skyline

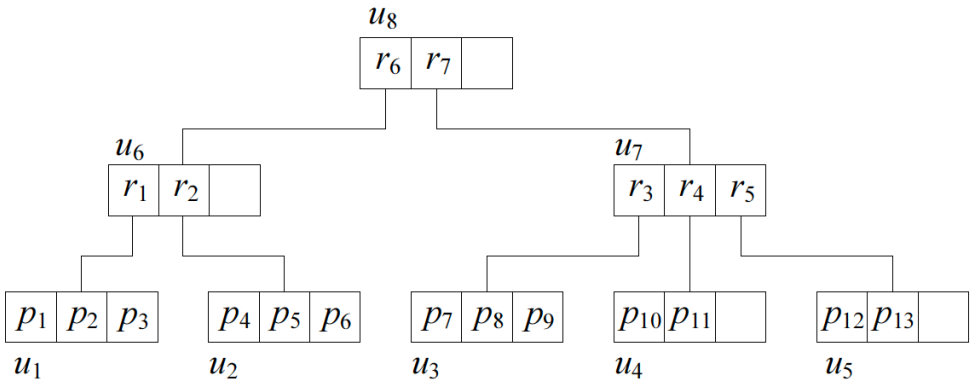
BBS

Remove from $H$ the next point $p_9$, which is the second skyline point found.

$H$ is now $\{(r_4, \sqrt{40}), (p_7, \sqrt{45}), (r_2, \sqrt{61}), (r_1, \sqrt{65}), (r_5, \sqrt{82})\}$.

$(p_9, \sqrt{29})$



# Branch and Bound Skyline

BBS

The next MBR $r_4$ in $H$ can be directly discarded, because its lower-left corner is dominated by a skyline point $p_9$. This implies that no points in $r_4$ can possibly be in the skyline.

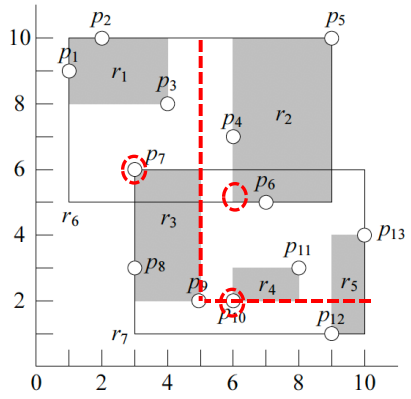
dominated by $p_8$

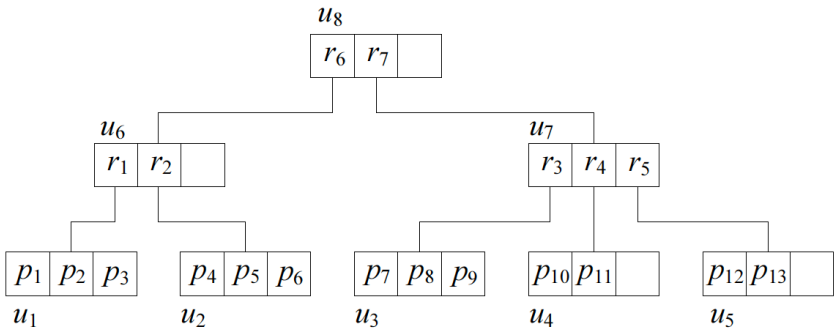
Following the same reasoning, both of the next point $p_7$ and the following MBR $r_2$ are also discarded.

Comparing the lower-left of MBRs with the corresponding coordinates of the current skyline points.

Currently $H = \{(r_1, \sqrt{65}), (r_5, \sqrt{82})\}$.
Skyline points found so far $= \{p_8, p_9\}$.

If an MBR is dominated, we do not need to access the points in the MBR, as all the inner points are dominated.

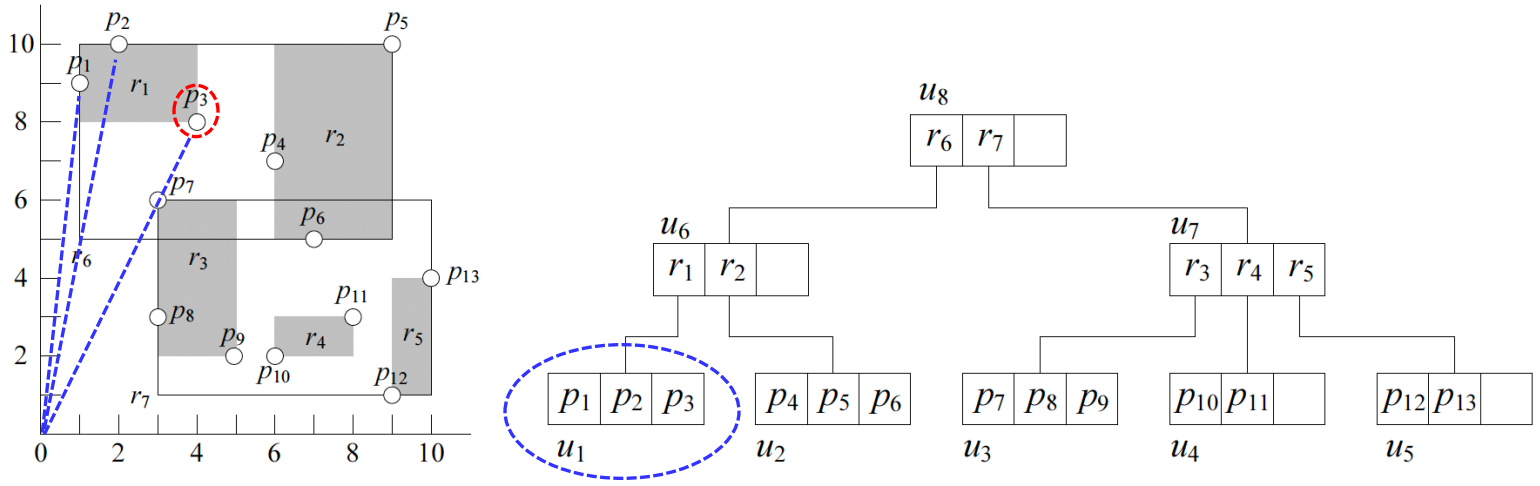


# Branch and Bound Skyline

BBS

Next, access the leaf node $u_1$, after which $H$ becomes $H = \{(p_3, \sqrt{80}), (p_1, \sqrt{82}), (r_5, \sqrt{82}), (p_2, \sqrt{104})\}$.

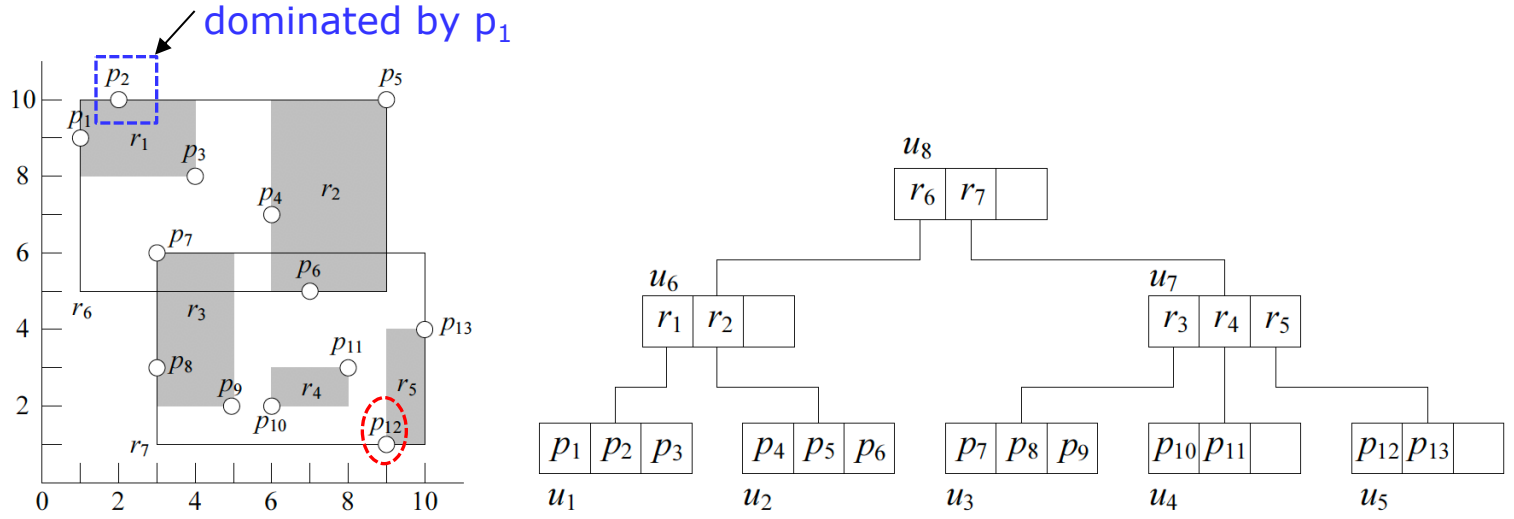



The next point $p_3$ is discarded. Then, we come to $p_1$, which is added to the skyline, which is now $\{p_1, p_8, p_9\}$.

# Branch and Bound Skyline

BBS

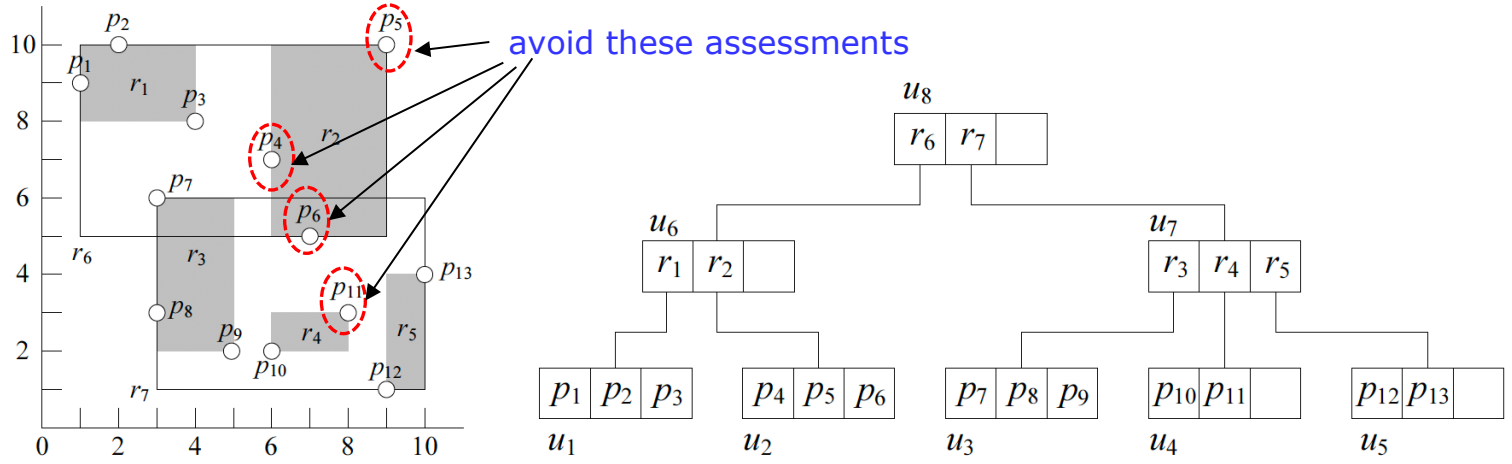
Now $H = \{(r_5, \sqrt{82}), (p_2, \sqrt{104})\}$. The rest of the algorithm proceeds in the same manner. We discover the 4th skyline point $p_{12}$ in $r_5$.



# Branch and Bound Skyline

BBS

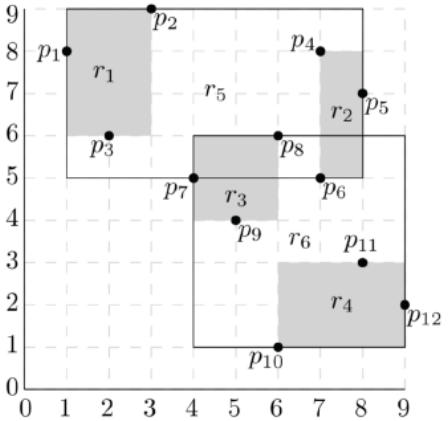
Now $H = \{(r_5, \sqrt{82}), (p_2, \sqrt{104})\}$. The rest of the algorithm proceeds in the same manner. We discover the 4th skyline point $p_{12}$ in $r_5$.

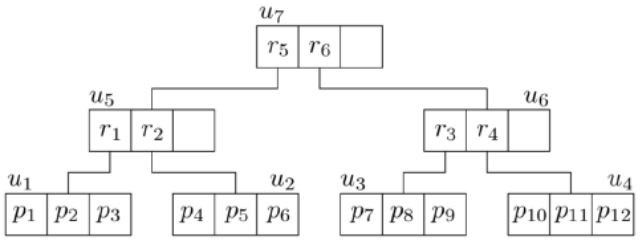


# Tutorial Questions

## Part 2: Skyline Search

All the following problems will be based on the dataset and R-tree below:




**Problem 1.** What is the skyline of the dataset?

**Problem 2.** Suppose that we use the NN algorithm to find the first skyline point, after which the NN algorithm will remove some points from the dataset. What are the *remaining* points in the dataset?

**Problem 3.** Indicate the nodes of the R-tree that are accessed by the BBS algorithm to find the skyline.

# Practical Exercises

Create an R-Tree structure to import the data points given in "***dataset_test***" under Week 10 workshop on iLearn.

- Suppose "***B=4***", you need to consider the lower bound and upper bound in this exercise.
- A function "***Choose Subtree***" should be devised to decide where should we insert a new data point.
- A function "***Handel Overflow***" should be devised, where the **perimeter sum** should be considered in splitting a node or an MBR.
- In the splitting, we need to consider the perimeter sum of several combinations of the splits.